

Plan de Proyecto: Mega App Updater

Aplicación de escritorio auto-actualizable para herramientas internas de la empresa. Distribución sin instalación de Python por parte del usuario final.

1. Visión General

Construir una aplicación nativa de Windows que:

- Agrupe múltiples herramientas internas bajo una misma UI (toolbar).
- Permita al equipo de desarrollo publicar nuevas herramientas y actualizaciones sin que el usuario final tenga que hacer nada manual.
- No requiera que los usuarios tengan Python, Node, ni ningún runtime instalado.
- Pueda ejecutar scripts Python existentes (ej. Excel → PowerPoint) internamente.

Usuario objetivo: <50 empleados internos, perfil no técnico.

2. Stack Tecnológico

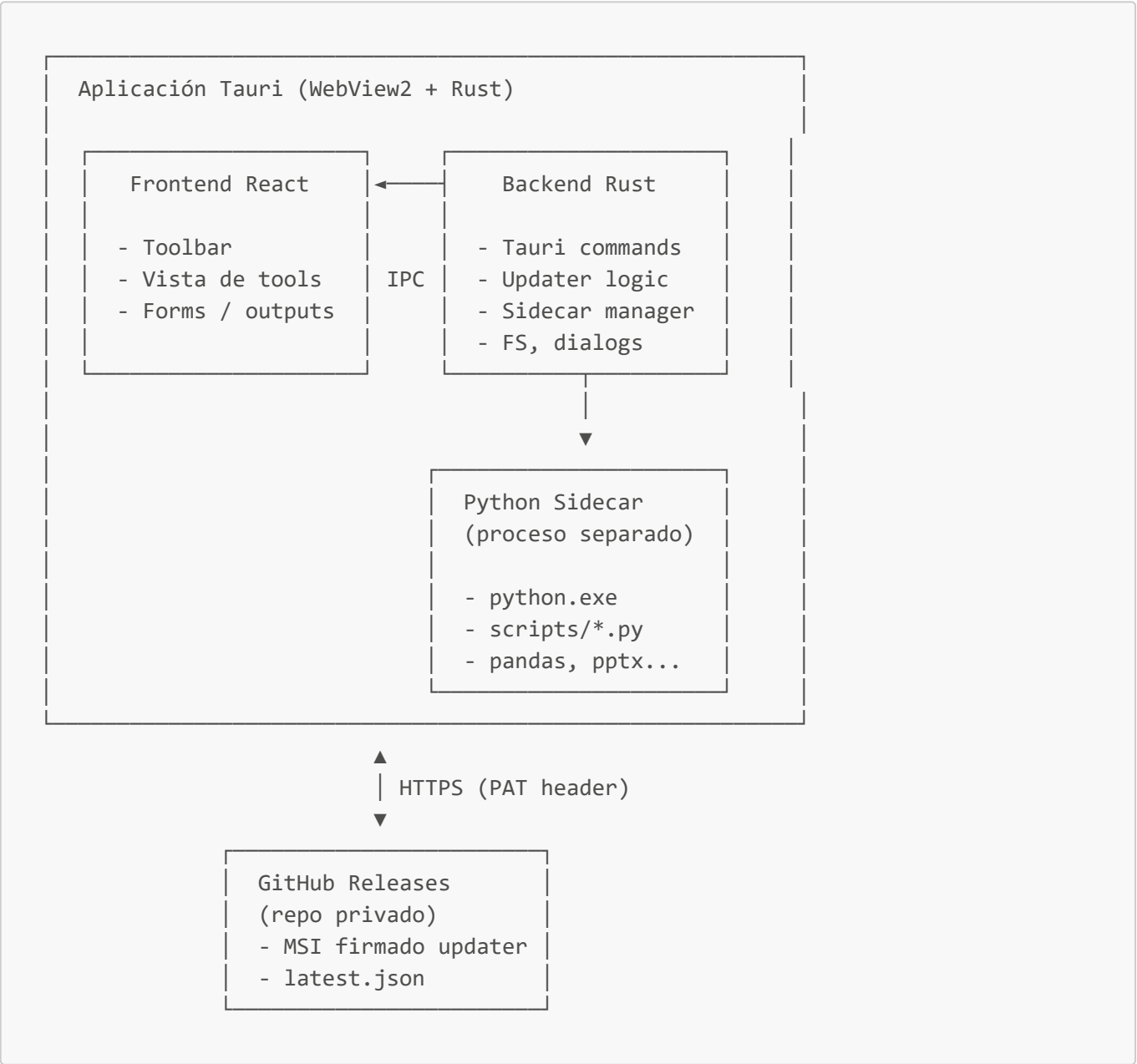
Capa	Tecnología	Justificación
Shell de la app	Tauri 2 (Rust)	Binarios chicos (~10 MB base), updater oficial, menos falsos positivos de antivirus que PyInstaller
Frontend	React 18 + TypeScript	Stack conocido por el equipo
Bundler	Vite	Estándar de Tauri. Next.js no aporta nada en una app de escritorio
UI components	shadcn/ui + Tailwind CSS	Rápido de iterar, moderno, totalmente customizable
Runtime Python	python-build-standalone	Python embebible sin instalación previa
Lógica Python	Scripts existentes + pandas , openpyxl , python-pptx	Reutiliza código actual
Updater	tauri-plugin-updater (oficial)	Integrado con Tauri, verificación criptográfica
Distribución	GitHub Releases (repo privado)	Ya tenemos acceso, no requiere infraestructura extra
Autenticación de updates	PAT embebido (read-only)	Simple, aceptable para uso interno
CI/CD	GitHub Actions (runner Windows)	Builds reproducibles y automáticos

Capa	Tecnología	Justificación
Plataformas soportadas	Windows 10/11 x64	Descartado Mac/Linux

Stacks descartados y por qué

- **Python + Flet + PyInstaller**: bundles de 150+ MB, falsos positivos de antivirus, updater hay que escribirlo a mano.
- **Electron**: ecosistema más maduro pero binarios 10x más grandes que Tauri.
- **.NET MAUI / WPF**: lock-in con el ecosistema Microsoft, curva de aprendizaje mayor para el equipo.
- **Next.js** (como frontend): innecesario en Tauri, no hay SSR ni API routes que aprovechar.

3. Arquitectura del Sistema



Comunicación Frontend ↔ Backend

- React llama a Rust con `invoke("nombre_comando", { args })`.
- Rust expone comandos con el macro `#[tauri::command]`.
- Eventos asíncronos (progress, logs) usan `app.emit("evento", payload)`.

Comunicación Backend ↔ Python Sidecar

- Rust ejecuta el sidecar como subprocesso (`tauri-plugin-shell`).
- **Protocolo:** argumentos por CLI + respuesta JSON por stdout, errores por stderr.
- Para operaciones largas, el script Python puede imprimir eventos JSON línea a línea (streaming) que Rust reenvía a React como eventos de progreso.

4. Python Sidecar: Estrategia

Build-time (una vez, o al cambiar dependencias)

1. Script PowerShell (`scripts/bundle-python.ps1`) descarga `python-build-standalone` versión Windows x86_64 (Python 3.12).
2. Extrae en `src-tauri/binaries/python-runtime/`.
3. Ejecuta `pip install --target ./lib -r python-scripts/requirements.txt`.
4. Copia los scripts `.py` a `src-tauri/binaries/scripts/`.
5. Renombra el `python.exe` según el target triple que Tauri espera (`python-runner-x86_64-pc-windows-msvc.exe`).

Bundle-time (cada release)

- `tauri.conf.json` declara el sidecar en `bundle.externalBin`.
- El MSI final contiene: app Tauri + runtime Python + dependencias + scripts.

Runtime (cuando el usuario usa una herramienta)

```

React clic "Generar PPT"
  |
  ▼
invoke("generate_pptx", { excelPath })
  |
  ▼
Rust command → tauri-plugin-shell.sidecar("python-runner")
                  .args(["scripts/excel_to_pptx.py", excelPath])
  |
  ▼
Python: lee Excel → genera PPTX → print(json)
  |
  ▼
Rust parsea JSON → devuelve a React
  |
  ▼
React muestra resultado / abre archivo

```

Tamaño estimado del bundle

Componente	Tamaño aprox.
Tauri app base	~10 MB
Python runtime standalone	~20 MB
pandas + numpy	~40 MB
openpyxl + python-pptx	~5 MB
Total MSI estimado	~70-80 MB

Si pandas no es imprescindible, bajar a ~35-40 MB.

5. Updater: Flujo Detallado

Firmas (clarificación importante)

Existen **dos firmas** distintas:

Firma	¿Obligatoria?	¿Cuesta?	Qué hace
Firma de Windows (Authenticode)	No, pero recomendada	~\$100-300/año	Evita SmartScreen warning
Firma Ed25519 del updater Tauri	Sí	Gratis	Verifica que el update vino de nosotros

→ **Decisión actual:** NO firmar con Authenticode (asumimos el SmartScreen warning en el primer install). **Sí firmar con Ed25519** (gratis y obligatorio para el updater).

Configuración `tauri.conf.json`

```
{
  "plugins": {
    "updater": {
      "active": true,
      "endpoints": [
        "https://api.github.com/repos/ORG/mega-app-updater/releases/latest"
      ],
      "pubkey": "<clave-publica-ed25519>",
      "windows": { "installMode": "passive" }
    }
  }
}
```

Secuencia al iniciar la app

1. App arranca
2. Plugin updater hace GET al endpoint con Authorization: Bearer <PAT>
3. Compara versión local (tauri.conf.json) vs tag_name del release
4. Si remote > local:
 - a. Descarga el MSI adjunto al release
 - b. Verifica firma Ed25519 contra pubkey embebida
 - c. Muestra diálogo "Hay una actualización, ¿instalar?"
 - d. Si sí: lanza MSI en modo passive, cierra app, reabre nueva versión
5. Si no hay update: continúa normalmente

Autenticación al repo privado

- Crear **fine-grained PAT** con:
 - Scope: solo este repo
 - Permisos: **Contents: Read-only**
 - Expiración: 1 año (renovar)
- Embeberlo en el binario como constante Rust (aceptamos el riesgo para uso interno).
- En caso de leak, revocar y generar uno nuevo (requiere republicar la app).

Rollback plan

- Si una versión queda rota en producción:
 1. Marcar el release como "draft" o borrarlo en GitHub.
 2. Publicar un nuevo tag con número mayor con el fix.
 3. Los usuarios que ya actualizaron a la versión rota: se arregla con el siguiente update.
 4. Los usuarios que aún no actualizaron: ya no ven la versión rota.

6. Estructura del Repositorio

```

mega-app-updater/
├── src/                                # Frontend React
│   ├── main.tsx
│   ├── App.tsx
│   ├── components/
│   │   ├── Toolbar.tsx
│   │   ├── ui/                        # shadcn/ui components
│   │   └── UpdateDialog.tsx
│   ├── tools/                        # Una carpeta por herramienta
│   │   └── excel-to-pptx/
│   │       ├── ExcelToPptxView.tsx
│   │       └── types.ts
│   ├── lib/
│   │   ├── tauri.ts                  # Wrappers de invoke
│   │   └── utils.ts
│   └── styles/
│       └── globals.css
└── src-tauri/                        # Backend Rust
  
```

```

├── Cargo.toml
├── tauri.conf.json
├── build.rs
├── src/
│   ├── main.rs
│   ├── commands/
│   │   ├── mod.rs
│   │   └── excel_pptx.rs
│   ├── python_bridge.rs
│   └── updater.rs
├── icons/
├── binaries/                                # (gitignored) sidecars compilados
│   ├── python-runner-x86_64-pc-windows-msvc.exe
│   └── python-runtime/
├── python-scripts/                          # Código Python fuente
│   ├── excel_to_pptx.py
│   ├── _shared/
│   │   └── io_helpers.py
│   └── requirements.txt
├── scripts/                                # Scripts de build
│   ├── bundle-python.ps1
│   └── prepare-release.ps1
├── .github/
│   └── workflows/
│       ├── ci.yml                          # Build en PRs (artifact)
│       └── release.yml                     # Release en tags
├── docs/
│   ├── USER_GUIDE.md                      # Guía para usuarios finales (instalación)
│   └── DEV_GUIDE.md                       # Guía para el equipo de desarrollo
├── .gitignore
├── package.json
├── tsconfig.json
├── vite.config.ts
├── tailwind.config.ts
├── components.json                        # shadcn/ui config
├── PLAN.md                               # (este archivo)
└── README.md

```

7. Workflow de Desarrollo y Release

Desarrollo local (día a día)

```

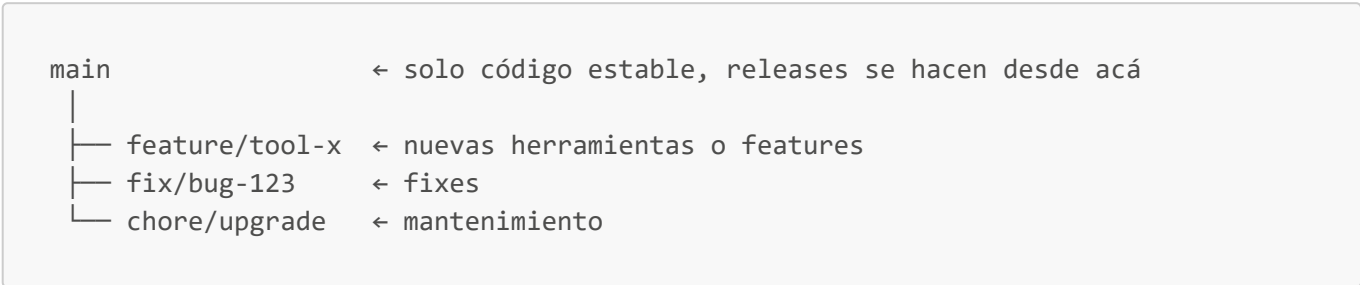
# Primera vez: bundle del Python sidecar
npm run bundle:python

```

```
# Development con hot-reload
npm run tauri dev
```

En modo dev, el frontend corre en Vite (<http://localhost:1420>) y Tauri lo embebe. El sidecar Python se ejecuta igual que en producción.

Branches y PRs



- Cada push a una feature branch → CI compila y deja un **MSI como artifact**.
- Los devs descargan el artifact y prueban la app completa antes de mergear.
- El updater **nunca** ve estos builds (no son releases con tag).

Proceso de release

1. Mergear PR a `main`.
2. Bump de versión en `src-tauri/tauri.conf.json` y `src-tauri/Cargo.toml`.
3. `git tag v1.2.0 && git push --tags`.
4. GitHub Action `release.yml` se dispara:
 - Compila app con sidecar Python.
 - Firma el MSI con la clave Ed25519.
 - Crea un GitHub Release con el MSI y el `latest.json`.
5. Próxima vez que cada usuario abra la app → se actualiza automáticamente.

Secrets de GitHub Actions necesarios

Secret	Descripción
TAURI_SIGNING_PRIVATE_KEY	Clave privada Ed25519 para firmar updates
TAURI_SIGNING_PRIVATE_KEY_PASSWORD	Password de la clave
UPDATER_GITHUB_TOKEN	PAT para embeber (puede ser el mismo que usa el workflow)

8. Roadmap por Fases

Fase 1 — Scaffold del proyecto (1-2 días)

- ☐ `npm create tauri-app@latest` con template React + TypeScript + Vite
- ☐ Configurar Tailwind CSS
- ☐ Inicializar shadcn/ui (`npm shadcn@latest init`)
- ☐ UI mínima: ventana + sidebar + vista placeholder

- ☐ Verificar compilación en Windows (`npm run tauri build`)
- ☐ Commit inicial

Fase 2 — Python sidecar (2-3 días)

- ☐ Script `bundle-python.ps1` descargando python-build-standalone 3.12
- ☐ `requirements.txt` con deps del script (openpyxl, python-pptx, pandas si aplica)
- ☐ Instalar deps al bundle
- ☐ Configurar `externalBin` en `tauri.conf.json`
- ☐ Command Rust de prueba que ejecute un `hello.py`
- ☐ Verificar que en el MSI buildado el Python se ejecuta correctamente

Fase 3 — Primera herramienta: Excel → PPT (1-2 días)

- ☐ Portar script existente a `python-scripts/excel_to_pptx.py`
- ☐ Adaptar: recibir path por argv, devolver JSON por stdout
- ☐ Vista React con input de archivo + botón
- ☐ Command Rust que orquesta el flujo
- ☐ Manejo de errores y feedback visual
- ☐ Abrir el archivo generado al terminar (usando `tauri-plugin-opener`)

Fase 4 — Auto-updater (1-2 días)

- ☐ Generar keypair: `npx tauri signer generate -w ~/.tauri/mega-app.key`
- ☐ Pubkey en `tauri.conf.json`, privkey como secret de GitHub
- ☐ Configurar endpoint apuntando a releases privados con header `Authorization`
- ☐ Probar update end-to-end: instalar v0.1.0, publicar v0.1.1, verificar update
- ☐ UI del diálogo de update (shadcn Dialog)

Fase 5 — CI/CD (1 día)

- ☐ Workflow `ci.yml` para PRs (compila + sube artifact)
- ☐ Workflow `release.yml` para tags (compila + firma + publica release)
- ☐ Configurar todos los secrets
- ☐ Primer release oficial v1.0.0

Fase 6 — Distribución (0.5 día)

- ☐ Escribir `USER_GUIDE.md` con screenshots del SmartScreen workaround
- ☐ Compartir MSI inicial con primeros usuarios (onboarding manual)
- ☐ Desde ahí, los siguientes updates son automáticos

Fase 7 — Iteración continua

- ☐ Agregar nuevas herramientas según pedidos del equipo
- ☐ Cada herramienta sigue el patrón: carpeta en `src/tools/` + script en `python-scripts/`
- ☐ Mantener changelog por release

Estimación total MVP (Fases 1-6): ~7-10 días de trabajo.

9. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
SmartScreen bloquea primer install	Alta	Medio	Documentar workaround con screenshots
Windows Defender marca el MSI	Media	Alto	Submit for analysis a Microsoft (gratis, 24-48h)
Antivirus corporativo bloquea sidecar Python	Baja	Alto	Coordinar con IT si pasa; whitelist por hash
PAT embebido se filtra	Baja	Bajo	Permisos read-only al repo; revocar y rotar
Versión rota publicada afecta a todos	Media	Alto	Plan de rollback; testing en artifact antes de tag
MSI muy pesado (> 100 MB)	Media	Medio	Evaluar si pandas es necesario; usar <code>--target</code> con <code>--no-deps</code>
Paths con unicode/espacios rompen el sidecar	Alta	Medio	Testing específico con paths tipo <code>C:\Users\José Pérez\</code>
Python-build-standalone deja de mantenerse	Muy baja	Alto	Es proyecto de Astral (uv, ruff), muy activo

10. Decisiones Tomadas

- **Plataforma:** solo Windows 10/11 x64.
- **Stack:** Tauri 2 + React + Vite + shadcn/ui + Python sidecar.
- **Firma Windows:** no, asumimos SmartScreen warning inicial.
- **Firma updater:** sí, Ed25519 (gratis y obligatoria para Tauri updater).
- **Auth al repo privado:** PAT embebido con scope mínimo.
- **Canal beta:** no, pero CI genera artifacts para testing previo a release.
- **Python version:** 3.12.
- **Primer caso de uso:** Excel → PowerPoint.

11. Decisiones Pendientes

- ☐ Nombre definitivo de la app (aparece en título de ventana y menú inicio).
- ☐ Icono de la app (.ico de 256x256 con múltiples resoluciones).
- ☐ Revisar el script Python actual para saber deps exactas.
- ☐ ¿La app tiene configuración persistente? (si sí, usar `tauri-plugin-store`).
- ☐ ¿Logs locales para debugging? (recomendado: `tauri-plugin-log`).
- ☐ ¿Telemetría de uso? (no, por ahora).

12. Referencias

- [Tauri v2 docs](#)
- [Tauri Updater Plugin](#)
- [Tauri Shell Plugin \(sidecars\)](#)
- [python-build-standalone](#)
- [shadcn/ui](#)
- [Vite](#)